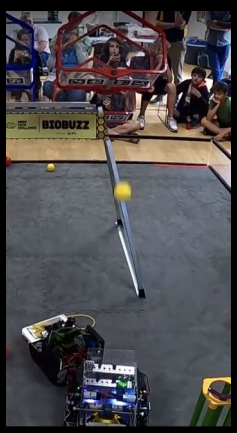


On screen while people sit down. No talking needed.

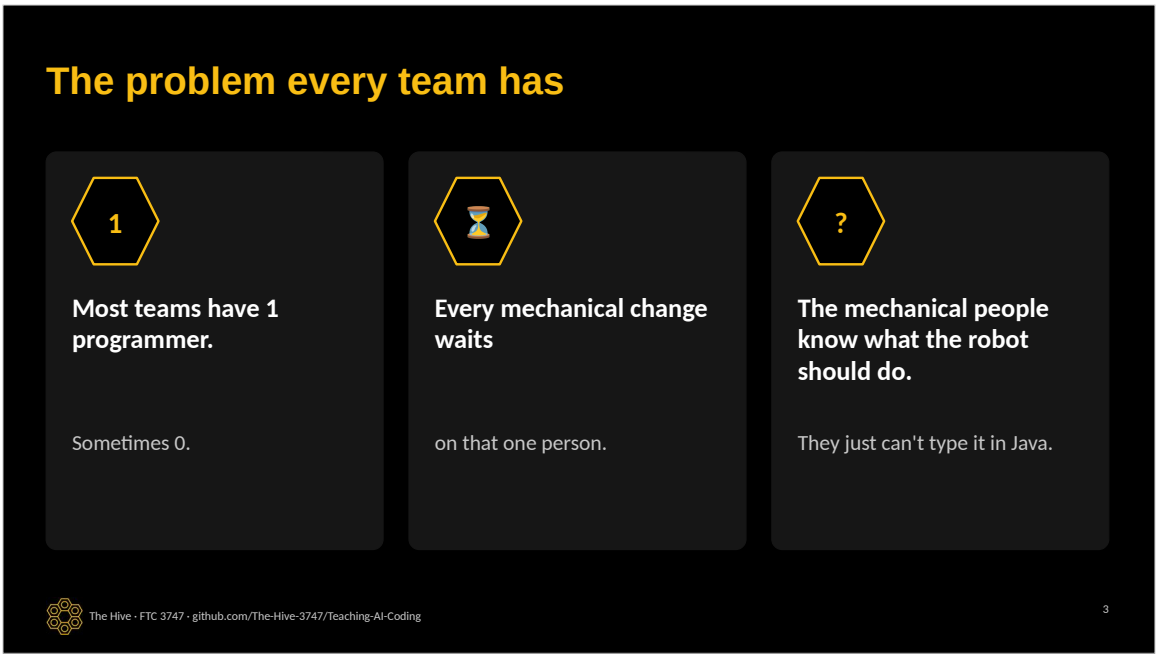
Who we are, and what this is

1. We're The Hive, FTC 3747. This year our students built and coded our robot by hand. No AI.
2. Our mentors entered the mentor competition with a separate robot, and ran an experiment on it: no hand-written Java. Everything described to Gemini.
3. We watched both. This is what the experiment showed, what we'd do with it, and how you can use it.

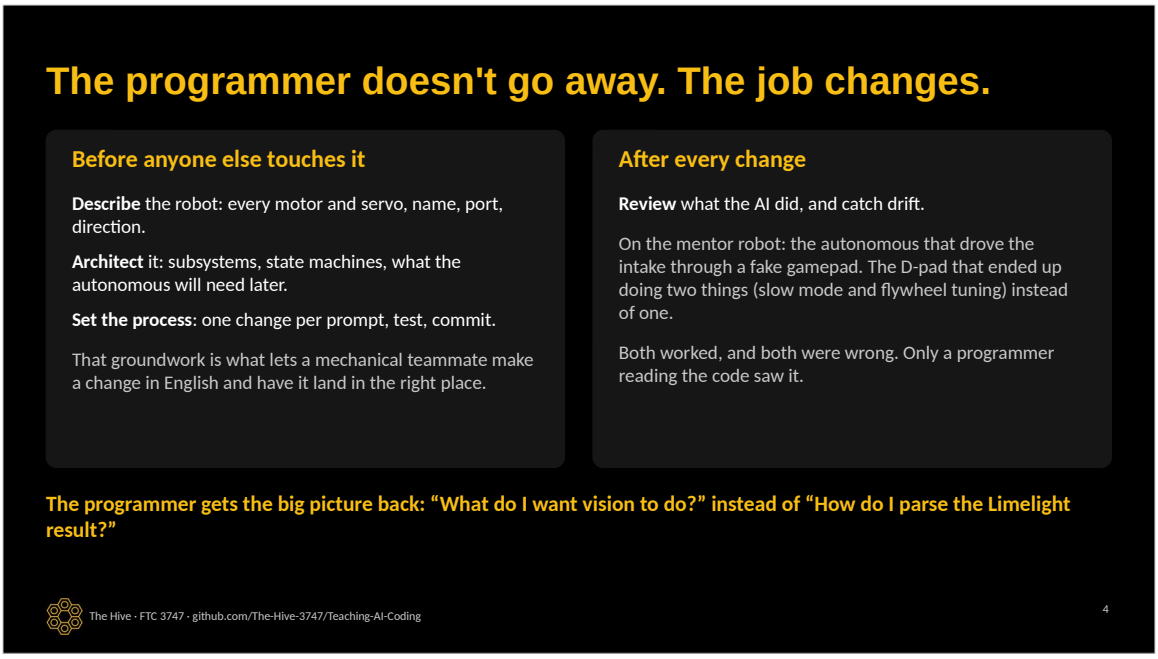


The mentor robot in a match

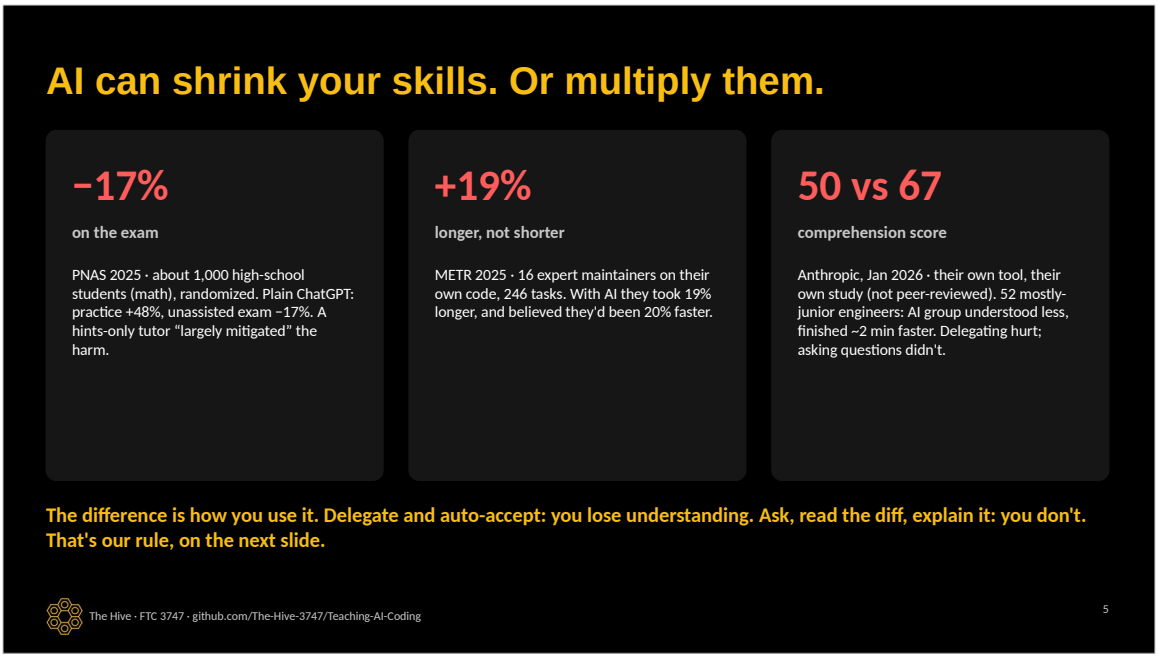
Say every line plainly. This room suspects mentors do our work. Line 1: our students hand-coded our robot this year, with no AI. Line 2: the AI experiment was the mentors', on a separate robot, for the mentor competition. Line 3: we're here to hand over what it showed. Don't argue the rumor. State the facts and move on.



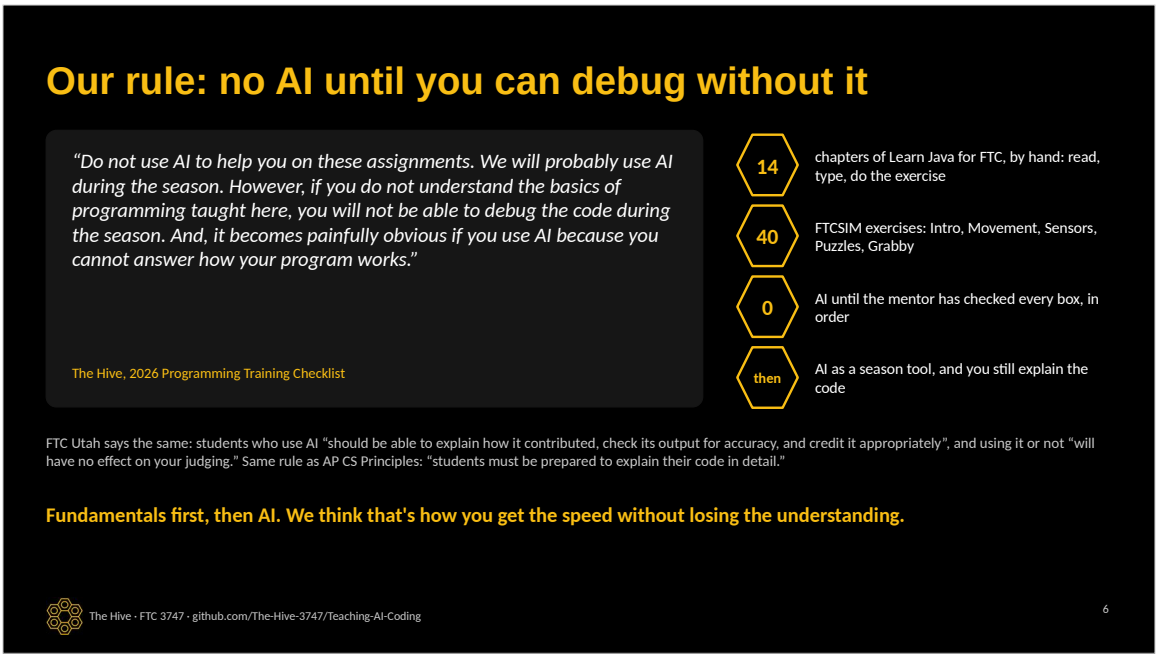
Ask for a show of hands: "How many teams have exactly one person who can write Java?" Then: "How many of those people are also the coach?" Let it land. This is the pain the method solves. If you want, say which of us is the one programmer on The Hive.



This is the answer to 'AI replaces the programmer'. It doesn't. The programmer moves up. The programmer does the prep that makes the mechanical teammate's English land: the description, the architecture, the process. Then they review every change for drift. The two real examples are on the slide, both from the mentor robot and both verbatim in the transcript: the autonomous that drove the intake through a simulated gamepad because nobody said an autonomous was coming, and the D-pad that was doing slow mode and flywheel tuning at once ('I thought gamepad 1 dpad was slow mode, not tuning the flywheel'). What the programmer gets back is the big picture: on our robot, adding one autonomous state was 10 to 20 minutes of copying boilerplate. That time becomes 'what do I want vision to do,' not 'how do I parse the Limelight result.'



Say this before they do. These are three of the strongest negative results, including the vendor's own, and all three fit our position. PNAS 2025: about 1,000 high-school students, math, randomized; plain ChatGPT made practice 48% better and the unassisted exam 17% worse. A hints-only tutor 'largely mitigated' the harm. METR 2025: 16 expert open-source maintainers on their own repos, 246 real tasks. With AI they were 19% slower and believed they'd been 20% faster, so the group that gains least couldn't even perceive it. Anthropic Jan 2026: their own tool, their own study, vendor-authored and not peer-reviewed, found harm against their own interest. 52 mostly-junior engineers learning a new library: the AI group scored 50% vs 67% on comprehension and was only about 2 minutes faster. 'Delegation' hurt. Asking conceptual questions didn't. What separates the harm studies from the no-harm studies is HOW the tool was used: delegate and auto-accept, versus ask, read the diff, explain it. That is our rule on the next slide. If someone



This is our onboarding checklist, verbatim. Read the whole quote out loud. It is the skeptics' own position, already in our program. Before a Hive programmer is cleared, the mentor checks off, in order, 14 of the first 15 chapters of Learn Java for FTC, each read-and-code plus the exercise, typed into our own simulator with a real test harness, and then 40 FTCSIM exercises. The book has 25 chapters: section 9, analog sensors, is skipped, and 16 to 25 are extra credit. Then, and only then, AI as a season tool. Say 'we think' on the link between this and the studies. Nobody has tested the sequence. It is our reading of them. College Board already requires the same thing for AP CS: students must be prepared to explain their code in detail. And at FTC judging the students answer (an adult may attend but may not interact or coach), so a team that can't explain its code loses the Control Award whether or not AI wrote it. FTC Utah's own statement (PDP email, Oct 2026) is the same rule in their words: no effect on judging either way, and students who use AI should

Seventy years of “that's not real programming”

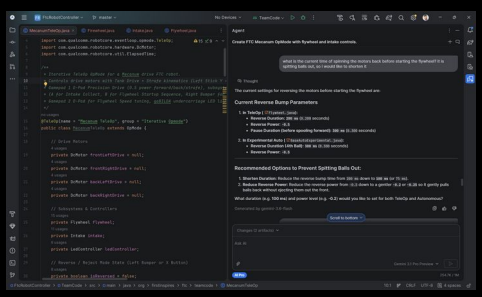
1954	FORTRAN	*John Backus, who led the FORTRAN team, writing in 1978: “the strength of the skepticism about ‘automatic programming’… as it existed in 1954”*
1983	Pascal	*Ed Post: “Real Programmers use FORTRAN. Quiche Eaters use PASCAL.”*
2005	IDE autocomplete	*Charles Petzold, on IntelliSense: “And I think it's making us dumber.”*
2017	Stack Overflow	15.4% of 1.3 M Android apps contained security-related code copied from Stack Overflow, and 97.9% of those had an insecure snippet (Fischer et al., IEEE S&P)
2026	AI coding	Gemini drove our mentors' autonomous through a fake gamepad. It worked, and a human reading the code caught it.

Every layer took the typing. None took the architecture, the testing, or the reading. Fred Brooks, 1987: “the hard part of building software [is] the specification, design, and testing of this conceptual construct, not the labor of representing it.”

Most new layers of programming tools were called a crutch by the generation before. In the sources we found there's no named 1954 skeptic, only Backus remembering them, so say 'most', not 'every'. Each layer took over the typing. None took over deciding what to build, testing it, or reading what came out. Brooks said it in 1987. Fowler's honest limit (2025) is that this layer is non-deterministic. The same prompt doesn't give the same code twice, so reading and testing matter MORE than before, not less. Then tell the fake-gamepad story: Gemini drove our mentors' autonomous through a simulated gamepad because nobody said an autonomous was coming. It worked, and a human reading the code caught it. Sources are in docs/research-ai-skills-case.md, slide 1 table.

What the mentors did that week

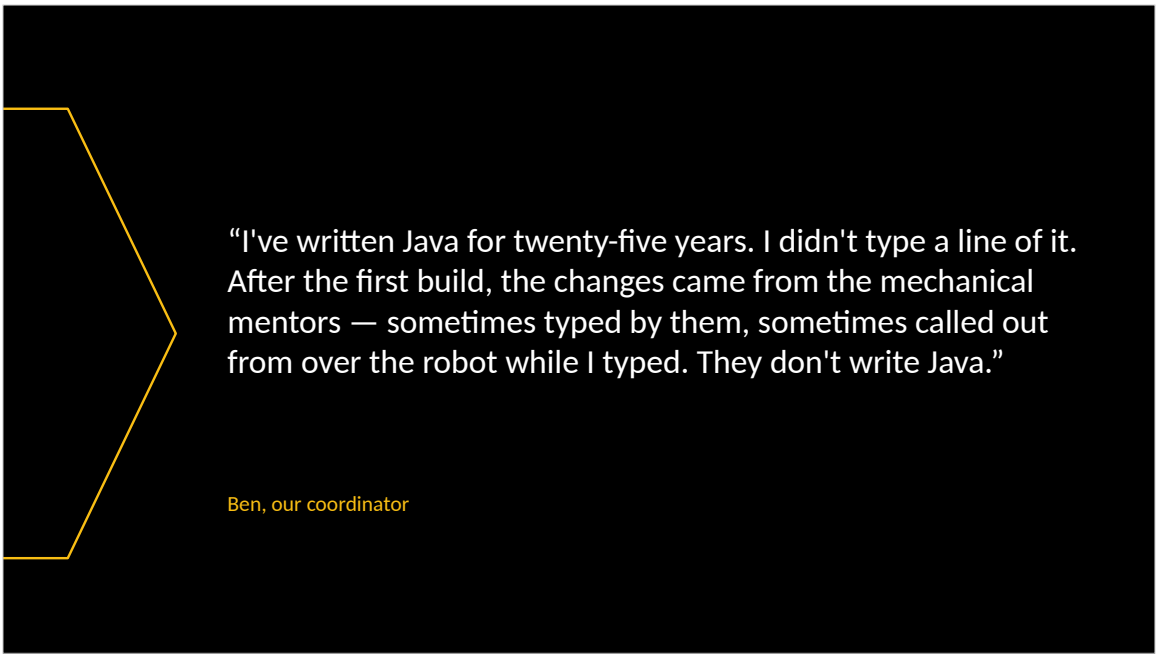
Sep 17, 10:26 PM	Ben (coordinator) describes the whole robot → working TeleOp that night, subsystem classes (25 minutes, 5 prompts)
Sep 18, ~9:50 PM	First autonomous state machines, Shoot First / Shoot Delayed. Asim's design, called out at the robot, typed by Ben
Sep 19, 8:29 AM	Asim (mechanical), at the keyboard: "holding both joysticks left made the robot strafe right, and vice versa. please fix"
8:36 AM	*"it is spitting balls out, so i would like to shorten it" (reverse bump 200 → 100 ms)*
8:40 AM	*"we get points for not touching the wall. Can we add a backup after the turn and drive forward?"*
11:35 AM	*"I thought gamepad 1 dpad was slow mode, not tuning the flywheel."*
11:37 AM	*"I don't want the intake to resume. I want the intake to stop (whether it was stopped or running)."*
2:57 PM	*"Let's set the default speed to 0.95" (default changed from 1.0)*
Scrimmage	Autonomous scored; TeleOp drove.



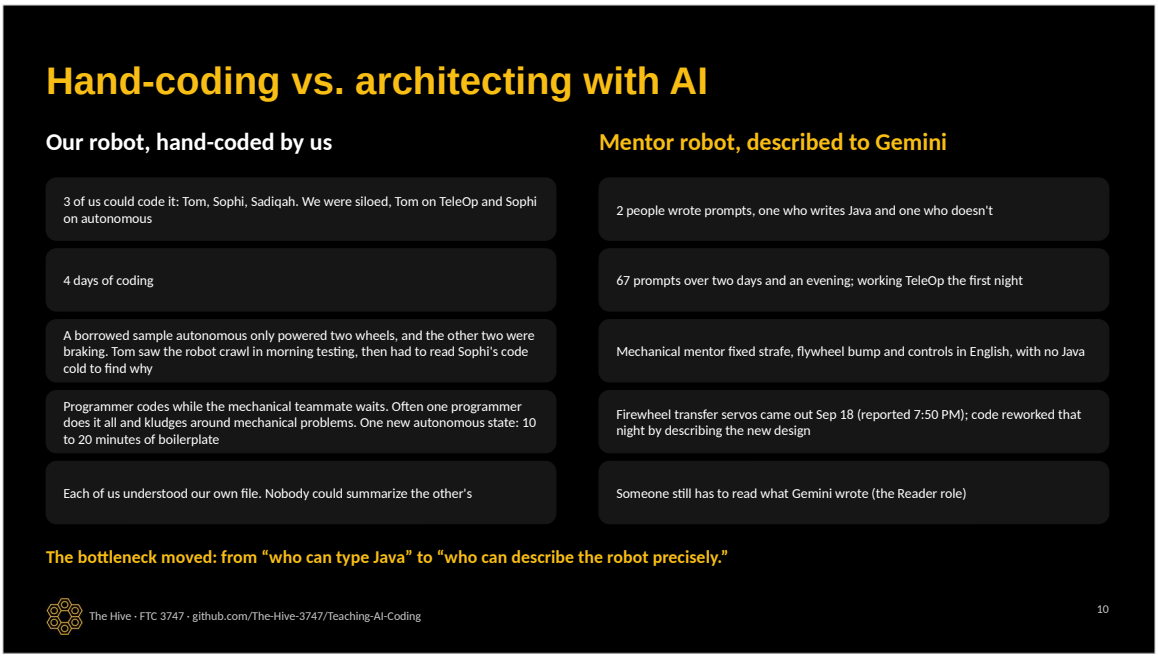
Sep 19, 8:36 AM, as typed, with Gemini's reply. It reports the numbers (200 ms at −0.5, 300 ms pause), offers two options, then changes both at once. The next prompt put the power back.

67 prompts, two days and an evening. 0 lines of Java typed.

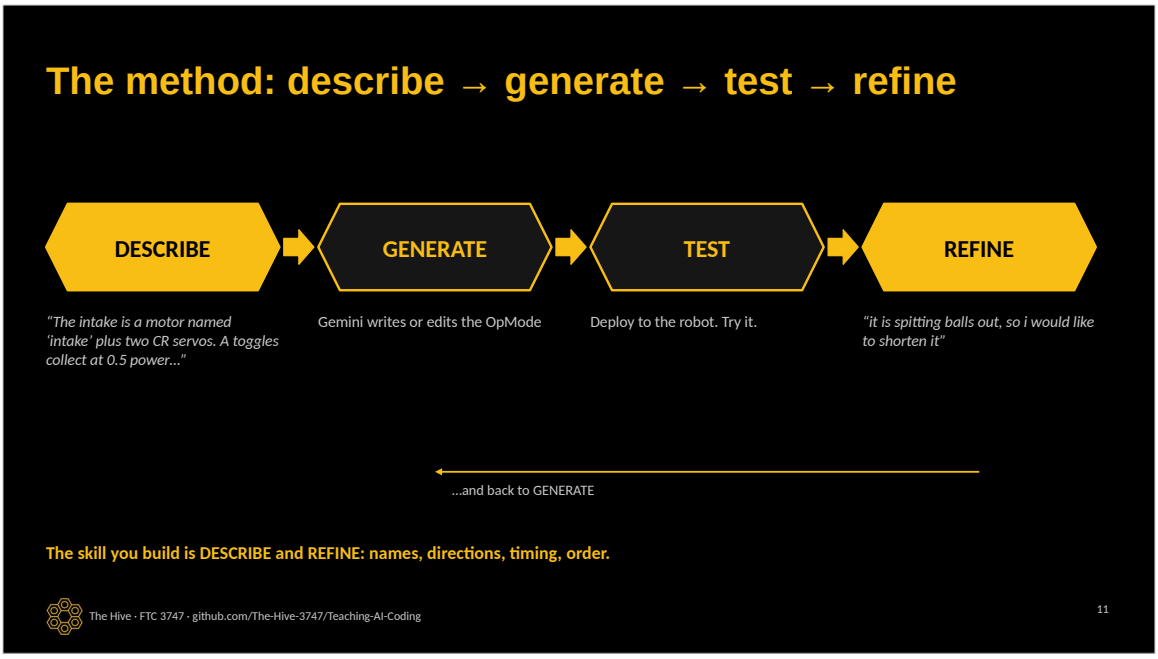
Tell this as a story, not a list. Be precise about who did what, because coaches will ask: Ben described the architecture on night one and built the TeleOp out over the first two days: the shooter startup sequence, the pulsed feed, the LEDs. The first autonomous was Asim's design, called out at the robot while Ben typed, and from there it was a pair, Asim deciding what to change and Ben typing, plus eight prompts Asim typed himself: tank drive, the strafe fix, the spitting-balls pair, a whole experimental autonomous with three rounds of tuning. The point for the room: the person who understands the robot drives the prompt, whoever's hands are on the keys. Two things make the experiment honest, and say both. Ben CAN write Java, so choosing to describe instead of type was a choice. Asim can't, and made most of the changes anyway. The quotes are verbatim, typos and all. The screenshot is the 'spitting balls out' prompt exactly as typed, with Gemini's answer. If there's time, play docs/video/scrimmage-match3-auto.mp4 from the laptop here.



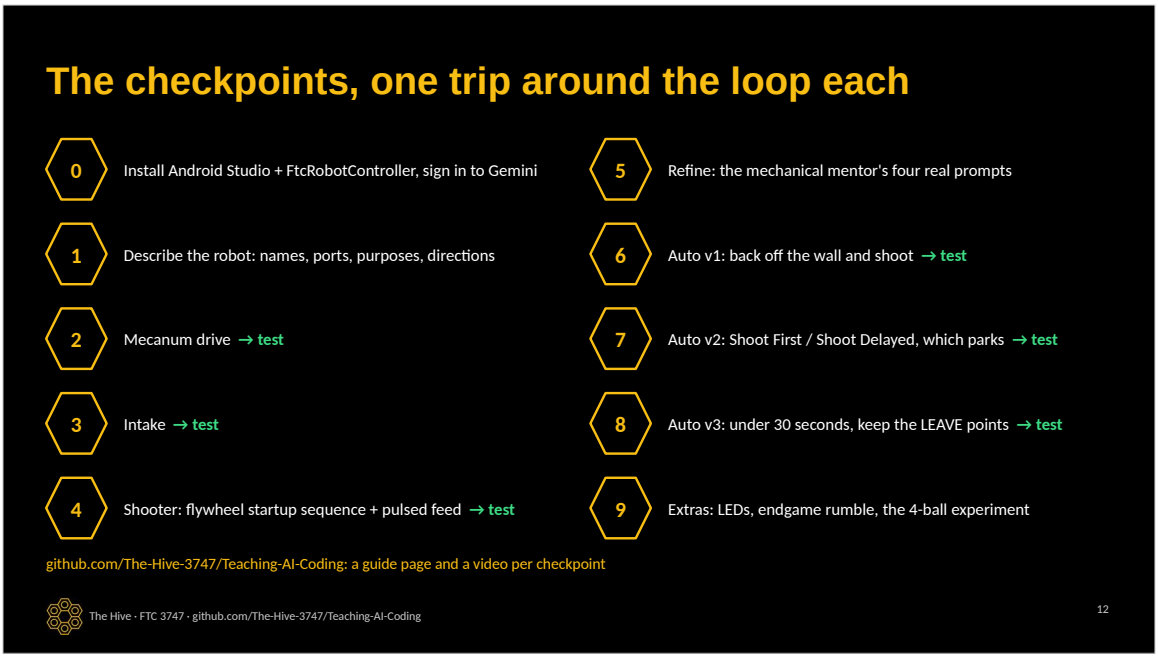
Put it up and let it sit for a few seconds. Then: the person who understands the robot drives the prompt, whoever's hands are on the keys.



This is the slide that's ours, and the left column is our real week. Four days of coding, three of us who could code the robot, and we were siloed: Tom wrote TeleOp, Sophi wrote autonomous. The bug: an autonomous borrowed from the FtcRobotController samples only powered two wheels, and the other two were set to brake at zero power, so the robot crawled. Tom saw it in testing the morning of the competition, then had to read Sophi's code cold, without knowing what she had done, to find why. Say the thing we'd do differently: AI could have given Tom a summary of the autonomous and he could have questioned it to find the bug. The grunt work: adding one state to the autonomous meant copying the same surrounding code again, 10 to 20 minutes for a new programmer, every time. Our process: the programmer works while a mechanical teammate waits, then sets up the robot and both watch the test. Mechanical problems get fixed if quick or go on the to-do list. Often it's one programmer doing all of it and kludging around



The loop is the whole workshop. Every checkpoint in the guide is one trip around it. The skill you build is DESCRIBE and REFINE: being specific about names, directions, timing and order.



Point out that 'test' comes after every step. The single biggest mistake is asking for everything at once. Small steps mean when something breaks, you know which description caused it. There's a guide page and a video per checkpoint. Show the repo URL.

What a good description has

1	**Names**	exactly as in the Robot Controller config: front_left_drive, not “the front left motor”
2	**Purpose**	what the part does for the robot
3	**Direction**	which way is “forward”; which motors are mounted mirrored
4	**Control**	which button or stick; toggle or hold; what happens on release
5	**Timing and order**	what has to happen before what, and for how long

Anything you leave out, Gemini guesses.

Read these out. Names go exactly as in the Robot Controller config: 'front_left_drive', not 'the front left motor'. If Gemini has to guess any of the five, it will, and the guess is where the bug comes from.

Vague vs. specific

Vague

"Add a shooter."

Which motor? Which way? Which button? Toggle or hold? What happens first?

Specific

"One motor named flywheel, reversed. Pressing B runs the flywheel and intake backward at −0.5 for 200 ms to clear a jammed ball, stops everything for 300 ms, then runs the flywheel forward at full power and keeps it there. Pressing B again stops it. While it's running and right bumper is held, pulse the intake 100 ms on at 0.5, 200 ms off, so balls feed one at a time."

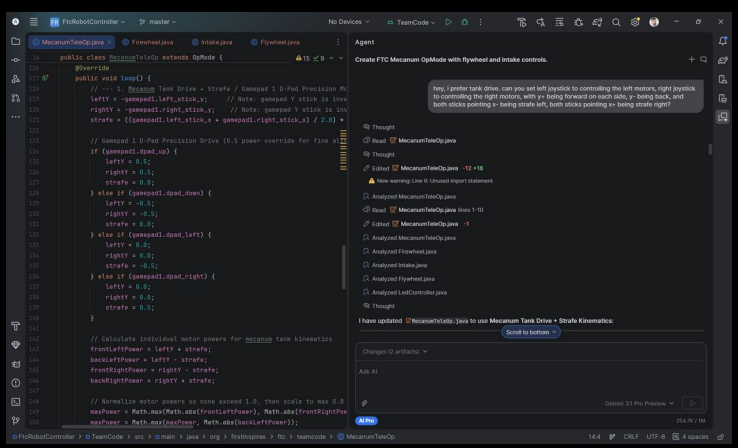
Real: Sep 19 session, verbatim

"When I start up the flywheel, I want the process to continue to do the reverse function. But then I don't want the intake to resume. I want the intake to stop (whether it was stopped or running). Does that make sense?"

No code words, just a sequence, and a few lines changed in Intake.java.

Read the vague one and ask the room what Gemini would have to guess. Then the specific one: everything it would have guessed is now stated. The third one is verbatim from the Sep 19 session. No code words at all, it describes a sequence, and that's why the fix was a few lines in Intake.java. This is the slide for the mechanical kids in the room.

Now watch it happen



The mentors' tank-drive prompt, as typed, and the code it produced. You're about to see the same thing live.

- Description already sent
- Drive prompt, live
- Read the code: names match
- Build, deploy, drive
- Refine: the real strafe fix
- Deploy, drive again

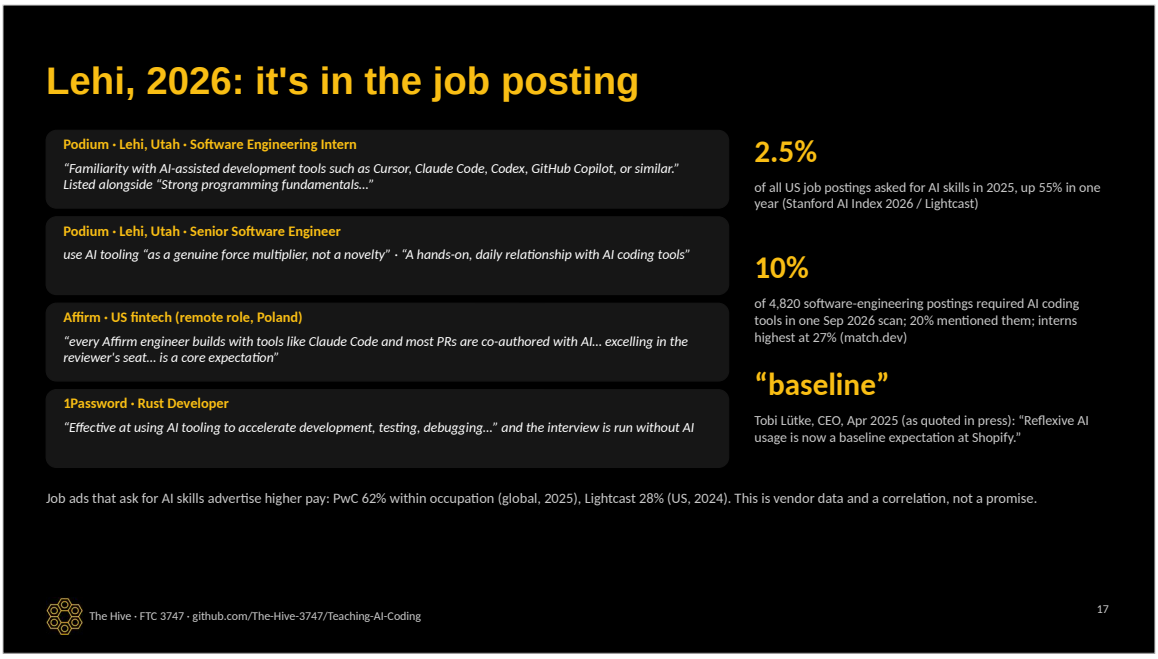
Switch to Android Studio. One of you drives the laptop, one narrates, one handles the robot. Virtual attendees are on Google Meet: share the Android Studio screen, and point a phone camera at the robot for the drive test. 1) Show the Gemini panel with the robot description already there. 2) Type the mecanum drive prompt live and send it; narrate what you asked for while it generates. 3) Show the generated OpMode and point at the hardware names, which match the config. 4) Build, deploy, drive. 5) Type the real refinement: "hey, in the latest push, holding both joysticks left made the robot strafe right, and vice versa. please fix", then deploy and drive again. 6) Stop there: "That's the whole method. Everything else is more trips around the loop." If it breaks, that's the method too: say what you see, paste the error into the panel, let the room watch the refine step. Don't apologize. A live fix is a better demo than a clean run.

Where it goes wrong, and the fix

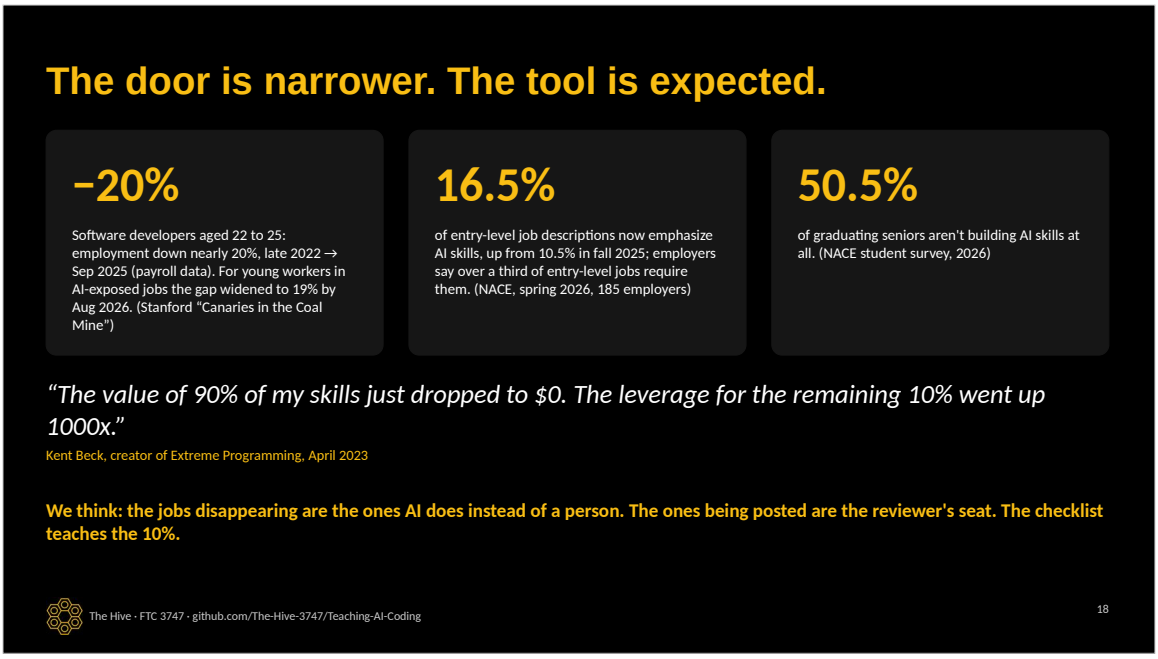
Symptom	Cause	Fix
Won't compile: “cannot find symbol”	Gemini invented a name	Paste the error back: “Fix this compile error”
Strafe is mirrored	Mecanum mixing signs	*“holding both joysticks left made the robot strafe right, and vice versa. please fix”*
Flywheel startup spits balls out	Reverse bump too long	*“what is the current time of spinning the motors back...? it is spitting balls out, so i would like to shorten it”*
Two controls on one button	Control scheme drifted	*“I thought gamepad 1 dpad was slow mode, not tuning the flywheel.”*
Auto drives the intake through a fake gamepad	Nobody told Gemini an autonomous was coming	Say it up front: “An autonomous will use this class later, so put the behavior in plain methods.” Gemini takes the expedient route unless you say what's coming.
Auto runs over 30 s	Gemini was told the limit; nobody saw the sum	“Add up every state's duration and tell me the worst-case run time.” Then cut one number.
Gemini rewrote everything, broke what worked	Didn't say “only change X”	“Only change the intake section. Leave the drive code as is.”

When it's wrong, the description was missing something. Say the missing thing.

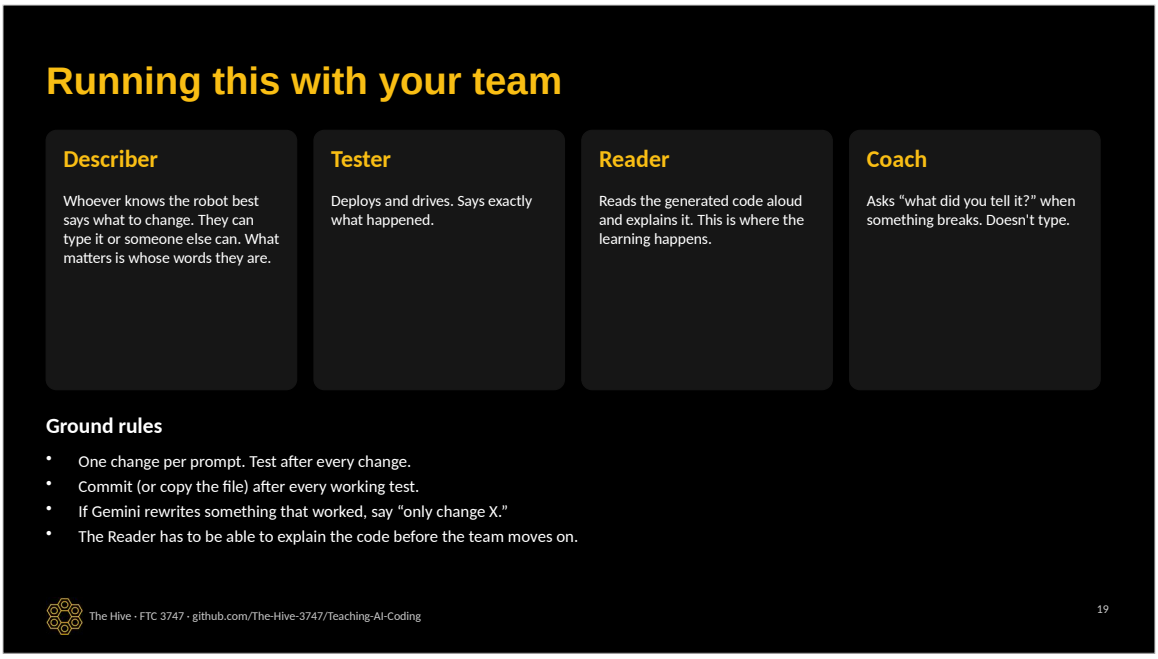
The italic rows and the fake-gamepad row happened on the mentor robot. The compile-error and over-30-seconds rows are the ones you'll hit first that the mentors got lucky on. Their delayed auto adds up to 30.7 s on paper and the field timer let it finish. The pattern: when it's wrong, the description was missing something. Say the missing thing. The fake-gamepad row is the one Ben cares most about: the auto worked, but it was tied to the TeleOp button map because nobody said an auto was coming.



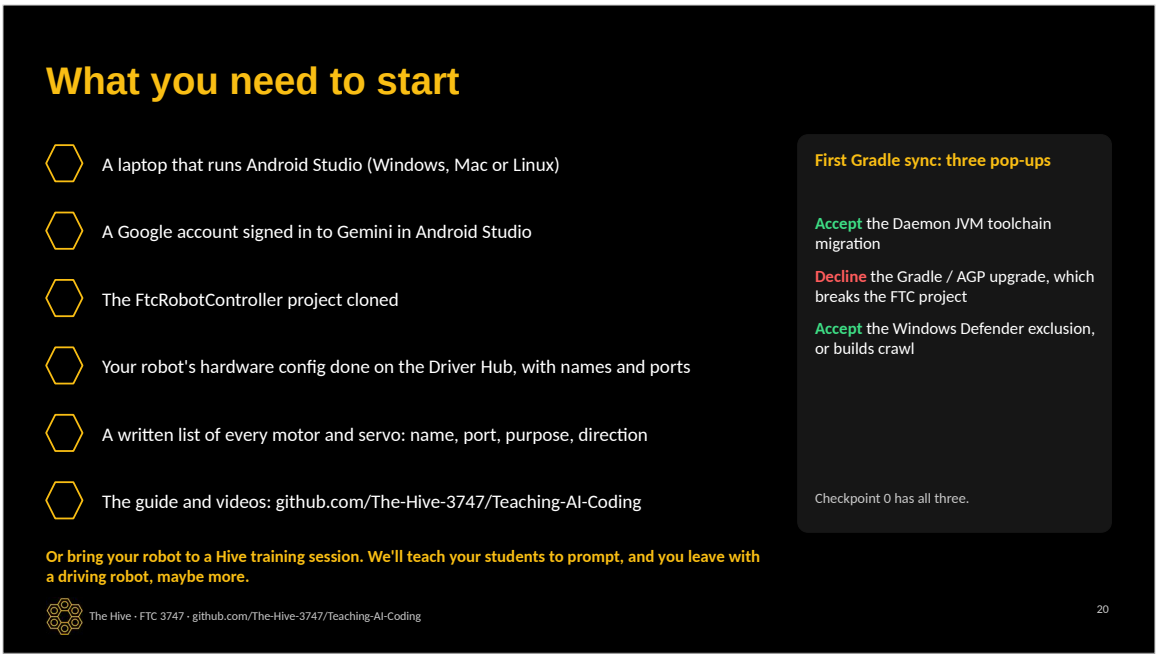
Lead with Podium. It's twenty minutes from here, it's an INTERN posting, and it names the tools. All four postings were verified live on Oct 3, 2026; re-open the links the morning of the talk, postings expire. Podium's senior posting uses our exact phrase: a 'genuine force multiplier, not a novelty.' Affirm's line is the sharpest: the job is now 'the reviewer's seat' (note: that posting is for a remote role based in Poland; Affirm is a US company). 1Password asks for both: use it daily, and interview without it. The pay numbers are vendor data and uncontrolled correlation, so say 'job ads advertise', never 'you will earn'. And NEVER say 'Lightcast 56%'. That number has no source. Shopify is quoted via press, not the memo itself. Don't use Microsoft's '66% won't hire without AI skills' (2024, self-reported, by the company selling Copilot).



Concede the first number completely. It's payroll data, the best in the room: software developers aged 22 to 25 are down nearly 20% since late 2022 (Stanford 'Canaries', Nov 2025, and the gap widened to 19% by Aug 2026), and the decline is where AI AUTOMATES work rather than augments it. Be precise about what it does NOT show: it does not show that having AI skills protects any individual kid. That half is our argument, so say 'we think'. The NACE numbers are a small self-reported employer survey (185 employers, sponsored by Jobscan), so lead with 16.5%, the harder number. The Kent Beck tweet is April 18, 2023, verbatim. This-season loss is an argument from one adult mentor experiment plus studies of professional developers, not evidence on students. Say that too if asked.



The Reader role is the answer to 'aren't they just cheating.' If a student can explain what the state machine does and why the timings are what they are, they learned it. Tell the room which of these roles each of you would take on The Hive. The Describer on the mentor robot was Asim, the mechanical mentor. He can't write Java, and he made most of the changes.



The hardware list is the one thing to do BEFORE opening Gemini. For anyone installing tonight: during the first Gradle sync, Android Studio pops up three things. Accept the Daemon JVM toolchain migration. DECLINE the Gradle/AGP upgrade. It breaks the FTC project. On Windows, accept the Defender exclusion or builds crawl. Checkpoint 0 in the guide has all three.

Who made this

Tom, Sophi and Sadiqah, The Hive's programmers

Hand-coded the team's own robot this season. Present this workshop and narrate the videos.

Ben, coordinator

Described the mentor robot's architecture to Gemini on night one. Typed no Java.

Annie, head coach

Asim, mechanical mentor

Built the mentor robot and authored most of the later changes in plain English, eight of them at the keyboard.

Royd, coach and mechanical engineer

Second coach on The Hive.

Claude (Anthropic)

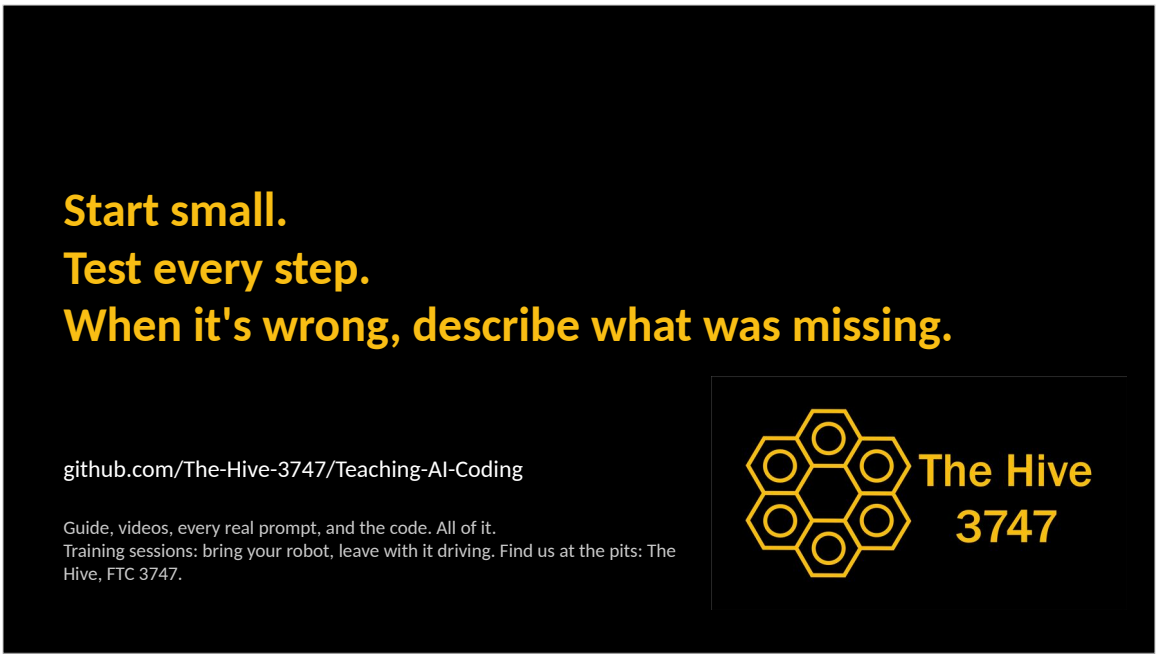
Compiled the transcript, guide and slides from the team's material.

Gemini in Android Studio (Google)

Wrote every line of the mentor robot's code from the mentors' descriptions.

Made the way it teaches: described by the team, generated, tested, refined.

Read the names. The point of the slide is that the people are named and the AI is named, the same way we're asking every team to credit its code. Royd is the team's second coach and a mechanical engineer.



No Q&A slot, so say where people can find you afterward.

Sources: studies, rules, history

Bastani et al., PNAS 2025
Generative AI can harm learning (~1,000 HS students) · scale.stanford.edu/publications/generative-ai-can-harm-learning

Becker et al., METR, Jul 2025
16 expert developers 19% slower with AI · arxiv.org/abs/2507.09089

Anthropic, Jan 2026
AI assistance and coding skill formation (vendor study) · anthropic.com/news/AI-assistance-coding-skills

FTC Utah: Team A.I. Use
PDP email, Oct 2026: no effect on judging; explain, check, credit · firstinspires.org/robotics/ftc

FTC AI Innovation Corner
FIRST Tech Challenge Docs · ftc-docs.firstinspires.org/en/latest/ai/innovation_corner/innovation-c

College Board, AP CS Principles
Create task directions: AI use and attribution · apcentral.collegeboard.org/media/pdf/ap-csp-student-task-directions.pd

FTC Competition Manual 2026-27
Awards section: A201, A204, A208 · ftc-resources.firstinspires.org/ftc/game/manual-06

FIRST blog: Expanding the toolbox with AI
Chris Rake, COO: "AI is just one more exciting addition to the toolbox" · community.firstinspires.org/expanding-the-first-toolbox-with-artificia

Learn Java for FTC (Alan Smith)
The textbook in The Hive's checklist · raw.githubusercontent.com/alan412/LearnJavaForFTC/master/LearnJavaForF

Backus, 1978
The History of FORTRAN I, II, and III (HOPL) · softwarepreservation.computerhistory.org/FORTRAN/paper/p165-backus.pdf

Post, 1983
Real Programmers Don't Use Pascal · ee.torontomu.ca/~elf/hack/realmen.html

Petzold, 2005
Does Visual Studio Rot the Mind? · charlespetzold.com/etc/DoesVisualStudioRotTheMind.html

Fischer et al., IEEE S&P 2017
Stack Overflow Considered Harmful? · arxiv.org/abs/1710.03135

Brooks, 1987
No Silver Bullet · worrydream.com/refs/Brooks-NoSilverBullet.pdf

Fowler, 2025
LLMs bring new nature of abstraction · martinfowler.com/articles/2025-nature-abstraction.html

Slides 5 to 7. Every title is a live link in the PDF and the .pptx. The caveats on each item are in docs/research-ai-skills-case.md; the 'must not repeat' table there is the one to read before you present.

Sources: employers and the market

Podium: Software Engineering Intern (Lehi)
verified Oct 3, 2026 · job-boards.greenhouse.io/podium81/jobs/7939921

Podium: Senior Software Engineer (Lehi)
verified Oct 3, 2026 · job-boards.greenhouse.io/podium81/jobs/6637207

Affirm: Senior Software Engineer
remote, Poland; verified Oct 3, 2026 · job-boards.greenhouse.io/affirm/jobs/7806681003

1Password: Rust Developer
verified Oct 3, 2026 · jobs.ashbyhq.com/1password/c247ea98-bf31-45f0-a38b-1fe255909538

Shopify memo, as quoted in Business Today
Apr 2025 (secondary) · businesstoday.in/technology/news/story/ai-use-is-no-longer-optional-at

Stanford AI Index 2026 / Lightcast
2.5% of US postings, +55% · lightcast.io/resources/research/stanford-ai-index-2026

match.dev, Sep 2026
AI coding tools in 4,820 SWE postings (one scan) · match.dev/post/ai-coding-tools-in-job-postings/data/

PwC Global AI Jobs Barometer 2026
62% wage premium (vendor, uncontrolled) · pwc.com/gx/en/issues/artificial-intelligence/job-barometer/2026/2026-g

Lightcast, Jul 2025
28% premium; 51% of AI postings outside IT (2024 data) · lightcast.io/resources/blog/beyond-the-buzz-press-release-2025-07-23

Brynjolfsson, Chandar & Chen, Nov 2025
Canaries in the Coal Mine (payroll data) · digitaleconomy.stanford.edu/app/uploads/2025/11/CanariesintheCoalMine_

Canaries dashboard
gap widened to 19%, Aug 2026 · digitaleconomy.stanford.edu/project/indicators/canaries-dashboard/

NACE, spring 2026
AI skills in entry-level jobs (185 employers) · naceweb.org/job-market/trends-and-predictions/demand-for-ai-skills-in-

NACE student survey, 2026
50.5% not building AI skills · naceweb.org/about-us/press/2026/ready-or-reluctant-employer-expectatio

Kent Beck, Apr 2023
90% of my skills are now worth $0 · tidyfirst.substack.com/p/90-of-my-skills-are-now-worth-0

The Hive: verbatim transcript and research report
every prompt, every number, with its caveats · github.com/The-Hive-3747/Teaching-AI-Coding

Slides 15 and 16 and the team's own records. Job postings expire: re-open the Podium links the morning of the talk. PwC and Lightcast sell AI-related services; their wage figures are advertised-pay correlations, not controlled comparisons.